OtimizadorNasem

Otimizador de **custo da dieta** de bovinos leiteiros por **Programação Linear (método Simplex / solver HiGHS)**, replicando as restrições nutricionais do padrão **NASEM** usadas no simulador RuFaS. Serve para escolher a dieta de menor custo que atende a todas as restrições nutricionais **antes** de rodar uma simulação completa no RuFaS.

É uma aplicação web em **Django + PostgreSQL**, localizada em `/root/OtimizadorNasem`.

---

1. Para que serve

O RuFaS tem um otimizador de ração interno (SLSQP, não-linear) que roda **durante** a simulação. O OtimizadorNasem é uma ferramenta **standalone** que resolve o mesmo problema (dieta de menor custo), mas:

- roda **sob demanda**, sem precisar simular a fazenda inteira;
- usa **Simplex** (solver linear HiGHS), que garante o ótimo global e fornece **preços-sombra** e **tableau** — informação que o SLSQP do RuFaS não entrega;
- permite **análise de sensibilidade** automatizada (variar preços, requisitos ou restrições e ver até onde a dieta ótima se mantém).

Em resumo: responde "qual a ração mais barata que ainda atende à nutrição exigida?" e "o que acontece com essa ração se o preço de um insumo mudar?".

---

2. Como funciona (a matemática)

Problema resolvido

- **Variáveis de decisão:** `x_i` = kg de matéria seca (MS) de cada alimento por vaca/dia.
- **Objetivo:** minimizar o custo `Σ (preço_i × x_i)`.
- **Sujeito a:** todas as restrições nutricionais do NASEM (energia, proteína, cálcio, fósforo, NDF, forage-NDF, gordura, consumo de MS).

A linearização (o "pulo do gato")

Várias restrições do RuFaS são percentuais da matéria seca total, o que é **não-linear**:

```
NDF% = Σ(NDF_i · x_i) / Σ(x_i) ≥ 25
```

Como `DMI = Σ x_i` é combinação linear das variáveis, multiplica-se os dois lados por DMI, eliminando a divisão:

```
Σ((NDF_i − 25) · x_i) ≥ 0      ← LINEAR
```

A mesma técnica lineariza: NDF mín/máx, forage-NDF mínimo, gordura máxima e as faixas de proteína.

A energia (alinhada ao RuFaS)

A energia é a única restrição genuinamente não-linear no RuFaS (depende do *intake discount*). O OtimizadorNasem usa a **cadeia de energia do RuFaS** de forma linearizada:

```
NE = 0,66 × (Σ DE_i·x_i − perdas_metano − perdas_urina)
```

onde:

- `0,66` = eficiência de uso da energia metabolizável → líquida (`EFF_OF_ME_USE`);
- `perdas_metano = 13,28 × metano(kg)`;

- `perdas_urina = 0,0146 × N_urinário(g)`.

> **Importante:** uma versão anterior comparava energia digestível bruta (DE) diretamente com o requisito de energia líquida (NE), o que subestimava o alimento necessário e gerava dietas baratas porém subnutridas. A correção (fator 0,66 + desconto de perdas) foi incorporada e as dietas agora entregam a energia líquida correta.

Limitação consciente

O *intake discount* (a eficiência da digestão cai conforme a vaca come mais) **não** é representável num modelo linear. Por isso a dieta do OtimizadorNasem fica ~20% mais barata que a do otimizador interno do RuFaS no mesmo ponto de operação. Essa diferença residual é estrutural do método Simplex, não um erro.

---

3. Onde as informações ficam salvas

Banco de dados

PostgreSQL **dedicado**, separado do `rufas_db`:

Item	Valor (default)
Banco	`otimizador_nasem`
Usuário	`superrufas` (`RUFAS_DB_USER`)
Senha	`superrufas123` (`RUFAS_DB_PASS`)
Host / porta	`localhost` / `5432`

As credenciais são lidas de variáveis de ambiente (as mesmas do RuFaS), com esses defaults.

Tabelas principais (schema `public`)

Tabela	Conteúdo
`feeds_feed`	Os 176 alimentos do NASEM (composição completa) + a coluna `price_per_kg` (preço por kg MS) + alimentos cadastrados pelo usuário.
`nutrition_animalprofile`	Perfis de animal (peso, leite, dias em lactação, etc.).
`nutrition_scenario`	Cenários de restrição (limites de NDF, gordura, DMI, flags de restrições).
`optimizer_optimizationrun`	Cada execução do otimizador (entradas, status, custo total, tableau).
`optimizer_optimizationitem`	A dieta ótima de cada execução (kg de cada alimento).
`optimizer_constraintresult`	Resultado de cada restrição (preço-sombra, folga, se está ativa).
`sensitivity_sensitivityrun` / `sensitivity_sensitivitypoint`	Análises de sensibilidade e seus pontos.

Origem da tabela NASEM

Importada do `NASEM_Comp.csv` (o mesmo arquivo que o RuFaS usa) pelo comando `import_nasem`. O identificador `rufas_id` é o mesmo `feed_type` do `feed.json` do RuFaS.

---

4. Como instalar e subir

```
cd /root/OtimizadorNasem

# Primeira vez: cria o banco, aplica migrações, importa o NASEM e cria os padrões
venv/bin/python manage.py bootstrap
```

```
# Sobe o servidor web
venv/bin/python manage.py runserver 0.0.0.0:8899
```

Acesse `http://localhost:8899/` (ou `http://IP_DO_SERVIDOR:8899/`).

> O `runserver` é o servidor de desenvolvimento do Django. Para produção, usar gunicorn/uwsgi + nginx.

---

5. O que preencher (passo a passo na interface)

A navegação segue a ordem do fluxo. Comece do topo:

Passo 1 — Alimentos e Preços

- Lista os 176 alimentos do NASEM.
- **Informe o `price_per_kg`** (R$/kg de matéria seca) dos alimentos que você vai usar. Sem preço, o alimento não entra na otimização.
- Pode buscar por nome/ID e filtrar por categoria.
- **Novo Alimento:** cadastra um ingrediente que não está no NASEM, informando toda a composição nutricional (PB, NDF, FA, Ca, P, DE, etc.). Recebe um `rufas_id` automático a partir de 1000.

Passo 2 — Parâmetros e Requisitos

Preencha os dados da **vaca-tipo** para a qual a dieta será otimizada. Os campos já vêm com os **valores padrão do RuFaS**:

Campo	Default	Unidade
Peso vivo	650	kg
Peso adulto	680	kg
Produção de leite	35	kg/dia
Gordura do leite	3,8	%
Proteína verdadeira do leite	3,1	%
Lactose do leite	4,85	%
Dias em lactação	120	dias
Parição	2	—
Escore de condição corporal	3,0	1–5
NDF da ração anterior	32	%
Metano entérico	400	g/dia
Nitrogênio urinário	0,20	kg/dia

Também é possível ajustar as **constantes das restrições** (NDF mín 25%, NDF máx 45%, forage-NDF mín 15%, gordura máx 7%, etc.) e ligar/desligar restrições individuais.

Ao salvar, o sistema **recalcula os requisitos** (energia, proteína, cálcio, fósforo, DMI prevista) pela metodologia NASEM e guarda os parâmetros para a otimização.

Passo 3 — Otimizar Dieta

- **Selecione o cardápio**: marque os alimentos disponíveis (precisam ter preço).
- Clique em **Otimizar**.
- O resultado mostra:
  - a **dieta ótima** (kg MS e % de cada alimento);
  - o **custo mínimo** (R$/vaca/dia);
  - os **nutrientes entregues × requeridos** (energia líquida NE, proteína, Ca, P, NDF, forage-NDF, gordura, DMI).

Se não houver dieta viável, o sistema avisa (ex.: cardápio insuficiente, faixa de DMI muito estreita).

Passo 4 — Tableau

A partir do resultado, abre o **tableau final do Simplex**:

- por alimento: valor na solução, custo reduzido, se está na base;
- por restrição: **preço-sombra** (quanto o custo muda se afrouxar aquele limite), folga, se está ativa (vinculante).
- Exportável em CSV.

Passo 5 — Análise de Sensibilidade

Varia um ou dois parâmetros numa faixa e reotimiza em cada ponto. Formatos aceitos no campo "parâmetro":

Formato	Exemplo	O que varia
`price:<rufas_id>`	`price:110`	preço de um alimento
`requirement:<chave>`	`requirement:protein_g`	um requisito (energy_mcal, protein_g, calcium_g, phosphorus_g, dmi_predicted)
`constant:<chave>`	`constant:min_forage_ndf`	uma constante (min_ndf, max_ndf, min_forage_ndf, max_fat, protein_upper_factor, dmi_constraint_fraction, dmi_requirement_boost)

Você informa início, fim e passo. O resultado traz:

- tabela de todos os pontos (custo, viabilidade, composição);
- **gráfico custo × parâmetro**;
- **intervalos de estabilidade**: a faixa em que a dieta ótima não muda de composição (não troca de "base").
- Exportável em CSV.

---

6. Como analisar os resultados no Metabase

O OtimizadorNasem tem interface web própria para ver os resultados (passos 3–5 acima). Mas os **custos da alimentação simulada pelo RuFaS** — para comparar com o ótimo — são analisados no **Metabase**, onde já existe a coleção dedicada.

Onde está

- Metabase: `http://localhost:3000`
- Coleção: **"RuFaS - Análise de Nutrição"**
- Dashboards:
  1. **Nutrição - Individual (por ID)** — filtro `run_id`
  2. **Nutrição - Comparação (A vs B)** — filtros `run_id_a` e `run_id_b`

Cards de custo (dashboard Individual)

- **A9 — Custo diário da dieta do curral** (US$/dia)
- **A10 — Custo diário por vaca** (US$/vaca/dia)
- **A11 — Decomposição do custo por alimento** (US$/dia)

Outros cards: consumo de MS por vaca, composição da dieta, participação (%) de cada alimento, eficiência alimentar (kg leite/kg MS), balanço nutricional e requisitos NASEM.

Como o custo é calculado no Metabase

Mesma lógica do OtimizadorNasem:

1. preços de cada alimento vêm do `run_input` (o `feed.json` gravado como JSONB), campo `purchased_feed_cost`;
2. consumo diário de cada alimento vem de `report_daily_ration_per_pen...LAC_COW.{feed_type}` (kg MS);
3. `custo = Σ(kg_ms × preço)`; dividido por `milking_cow_num` para o custo por vaca.

Como usar

1. Entre no dashboard **"Nutrição - Individual (por ID)"**.
2. Preencha o filtro **Run Id** com uma simulação, ex.: `GO_campo_dourado_14-Oct-2026_Wed_02-07-26`.
3. Veja os cards de custo e consumo preencherem.
4. Para comparar duas simulações, use o dashboard de **Comparação** com `run_id_a` e `run_id_b`.

> Liste os IDs disponíveis com:
> `SELECT run_id, farm_slug FROM simulation_run ORDER BY executed_at DESC;`

Comparar o ótimo com o simulado

A dieta **simulada** (no Metabase) usa proporções fixas definidas no `feed.json`; a dieta **ótima** (no OtimizadorNasem) é a de menor custo. É normal a ótima ser mais barata — esse é o ganho que a ferramenta mede. Na Campo Dourado, por exemplo, a dieta simulada custou ~US$ 6,18/vaca/dia e a ótima ~R$ 3,53, com o ótimo entregando exatamente a energia requerida (sem o excedente da dieta fixa).

---

7. Scripts de comparação (fora da interface)

Dois scripts auxiliares ajudam a validar o OtimizadorNasem contra o RuFaS:

Script	O que faz
`/root/Rufas/RuFaS/comparar_otimizadores.py`	Chama o otimizador **interno do RuFaS** (SLSQP) e o OtimizadorNasem com entradas idênticas, lado a lado. Mostra a diferença de custo e de energia (com e sem a correção de perdas).
`/root/OtimizadorNasem/comparar_com_simulacao.py`	Compara a dieta **real simulada** (extraída do Postgres) com a dieta ótima, em composição, custo e energia.

---

8. Comandos úteis

```
cd /root/OtimizadorNasem

# Reimportar a tabela NASEM (preserva os preços já informados)
venv/bin/python manage.py import_nasem

# Recriar os dados padrão (perfil e cenário)
venv/bin/python manage.py seed_defaults

# Rodar os testes
venv/bin/python manage.py test

# Acessar o admin do Django (CRUD completo das tabelas)
venv/bin/python manage.py createsuperuser   # cria o login
# depois acesse http://localhost:8899/admin/
```

---

9. Resumo das restrições (idênticas às do RuFaS — padrão NASEM)

Restrição	Limite padrão
Energia líquida (NE)	≥ requisito calculado
Proteína metabolizável	entre o requisito e 1,5× o requisito
Cálcio	≥ requisito
Fósforo	≥ requisito
NDF	entre 25% e 45% da MS

Restrição	Limite padrão
Forage-NDF	≥ 15% da MS
Gordura (ácidos graxos)	≤ 7% da MS
Consumo de MS (DMI)	faixa em torno da DMI prevista (−30% a +10%)

> O RuFaS **não** possui restrição de amido/concentrado máximo. O único limite indireto ao excesso de grão é o forage-NDF mínimo. Para uma dieta mais conservadora em grão, aumente a constante `min_forage_ndf` na tela de Parâmetros (ex.: 20%).